For any suggestions or feedback regarding these notes,
please contact [Pragy Agarwal]

Rate Limiting

Controlling the rate at which requests hit your system is fundamental to building stable, secure, and scalable applications.
Imagine launching a popular new service. Without controls, you might quickly encounter issues. Rate limiting addresses several critical needs…

- **Resource Protection**
  - Your servers (CPU, memory, database connections), downstream services, and network bandwidth have finite capacity.
  - Uncontrolled request volumes can exhaust these resources, leading to slowdowns, errors, or complete outages for everyone.
- **Abuse Prevention & Security**
  - Protection against Distributed Denial of Service (DDoS)
  - Preventing Credential stuffing (brute-force login attempts)
- **Fair Usage**
  - In multi-tenant systems or services with shared resources, rate limiting ensures no single client or user can monopolize those resources
  - Examples, internet FUP, ChatGPT rate limits
    - Unlimited data plan – up to 1Gbps FUP (up to 1TB)
- **Cost Management**
  - Cloud services, third-party API calls, and bandwidth incur costs based on usage.
  - Uncontrolled request rates can lead to unexpected and potentially huge bills.
- **Enforcing SLAs & Business Models**
  - Service tiers — enforcing quotas for Free vs Pro users

Type of Rate Limiters

We can classify rate limiting strategies along several axes:

- **By Location**
  - Where is the limit enforced
  - Client-Side, Server-Side (Gateway/Proxy/Middleware)

- **By Limiting Factor (Key)**
    - Based on what identifier
    - IP Address, User ID, API Key, Session ID, API Endpoint, Geographic Region, Global limit
- **By Algorithm**
    - The logic used to track usage and make allow/deny decisions
    - Fixed Window Counter
    - Sliding Window Log
    - Sliding Window Counter
    - Token Bucket
    - Leaky Bucket

Client-Side Rate Limiters

These primarily operate in the frontend to optimize UX and reduce unnecessary backend load initiated by the client.

Debouncing

When a user action triggers events very rapidly (eg, typing in a search suggestion box, resizing a window), and you only care about the *final* state after a pause. Making an API call on *every* keystroke is inefficient and wastes resources.

Solution

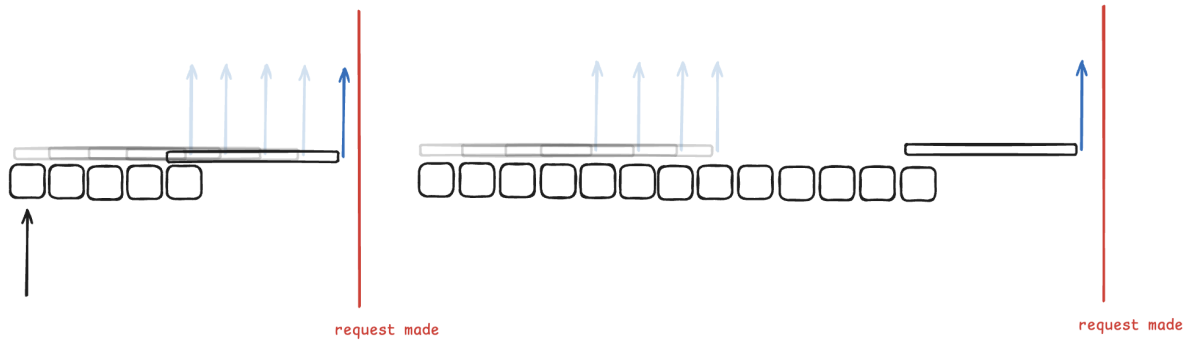
Implement a timer. When an event occurs, start (or reset) the timer for a short duration (eg, 300ms). If another event occurs before the timer finishes, reset the timer again. Only execute the target function (eg, make the API call) when the timer completes without being reset. Effectively, it waits for a pause in activity.

Example

An auto-suggest search bar only triggers the API call 300ms *after* the user stops typing. If they type rapidly, only one API call is made after they pause.

Pseudocode

```
function debounce(func, delayMilliseconds) {
    let timeoutId = null

    return function(...args) {
        const context = this           // Preserve 'this' context
        clearTimeout(timeoutId)        // Clear any existing timer
        timeoutId = setTimeout(() => {
            func.apply(context, args)  // Execute after wait time
        }, delayMilliseconds)
    }
}
// Usage
searchInput.addEventListener('keyup', debounce(fetchSuggestions, 300));
```



Note that in debouncing, there's no predetermined "rate" limit. There's only a Timeout which is enforced, but there's no way for you to know how many actual requests will come per second, as that will depend on when the user pauses.

Throttling

Need to guarantee a function isn't executed more often than a set interval (eg, `max 3 req/s`), no matter how often it's triggered. Useful for frequent events like scrolling or button mashing.

Solution

Allow execution on the first trigger, then ignore subsequent triggers during a cooldown period. After the cooldown, the next trigger is allowed again.

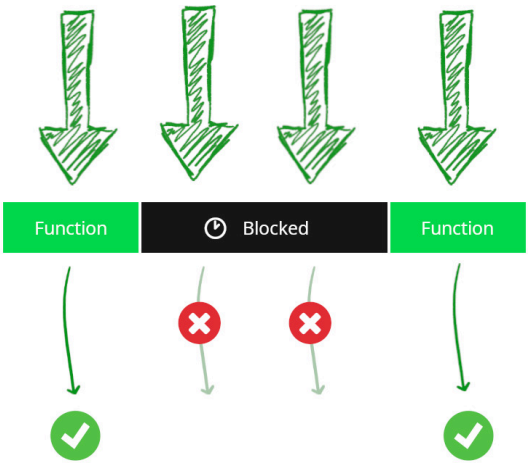
Example

Handle scroll events at most every `100ms`. Prevent multiple form submissions from rapid button clicks by throttling the handler to once every `2s`.

Pseudocode

```
function throttle(func, delayMilliseconds) {
    let coolingDown = false
    let lastArgs = null, lastThis = null

    return function(...args) {
        lastArgs = args; lastThis = this   // Preserve Context

        if (!coolingDown) {
            func.apply(lastThis, lastArgs) // Execute immediately
            coolingDown = true

            setTimeout(() => {
                coolingDown = false
```

```
                // Optionally execute again with latest args
                // if needed (trailing edge logic)
            }, delayMilliseconds)
        }
    }
}

// Usage:
window.addEventListener('scroll', throttle(processScrollPosition, 100));
```



Client-side Rate Limiting is *necessary* but *insufficient*.
You *always* need Backend Rate Limiting as well!

Backend Rate Limiters

On the backend, the enforcement logic can live in different places

- **In-Application Code**
  - Logic directly within service code.
  - Offers full context but adds complexity and makes global limits hard.
- **Middleware** (Framework Layer)
  - Integrated into web framework request pipelines
  - Better separation than raw application code.
- **Gateway/Reverse Proxy**
  - Implemented at the edge — Nginx, Kong, AWS API Gateway
  - Efficient for simpler limits (IP-based)
  - offloads application servers.

    - allows global rate limits

- **Dedicated Rate Limiter Service**
    - A separate microservice handling rate limit checks.
    - Centralizes logic but adds network latency.

For large companies, you want a layered approach (gateway + application-level) to prevent single point of failure.

Single point of failure here doesn't mean the server goes down – it means due to some code bug, your rate limits get bypassed.

Backend Rate Limiting Algorithms

Rate limiting algorithm decides if a request should be allowed or denied based on **configured limits** *(5 req/s, or 10 tokens per minute)* and **tracked state** *(limit by user_id, limit by ip_address, limit by api_key)*

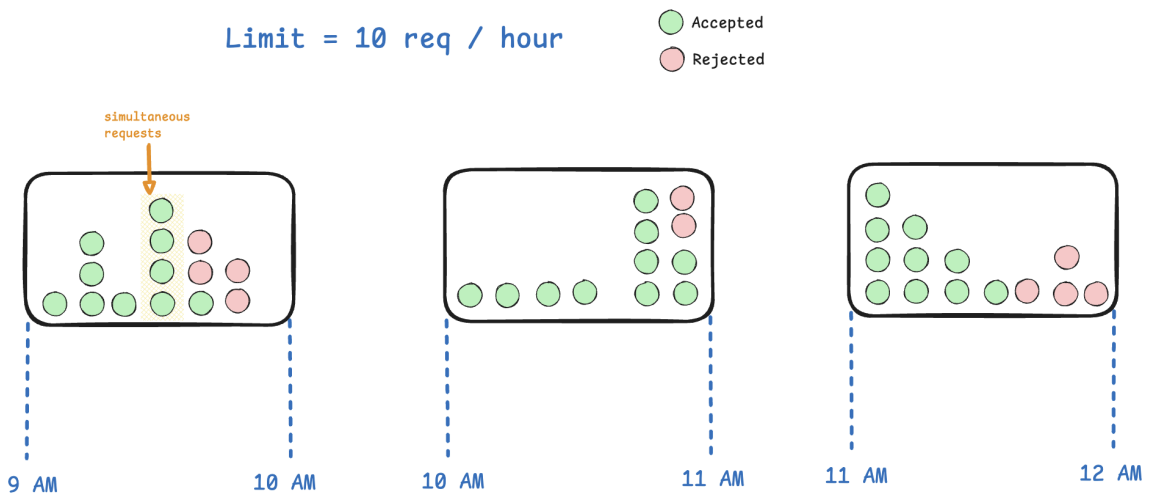
Fixed Window Counter

Simplest & fastest method to limit requests within fixed time intervals (like per second, per hour …)

Map<IpAddress, count>

Approach

- Time is divided into fixed windows (eg, `9:00` – `9:15`, `9:15` – `9:30`).
- A counter is maintained for each unique key (IP, UserID, API Key) within the *current* time window.
- When a request arrives
    - the counter for its key in the current window is incremented.
    - If the count exceeds the defined limit, the request is rejected;
    - otherwise, it's allowed.
    - Counters automatically reset to zero when a new time window begins
        - reset happens lazily – when the next request comes, we check if the previous window has expired.



Pseudocode

```
import time
import math

# setting rate limit to 10 requests / min
LIMIT = 10
WINDOW_SIZE_SECONDS = 60

# Assume STORE is a key-value store object with atomic incr() and expire() methods.
# like a Redis client
def is_allowed_fixed_window(key: str) -> bool:
    """Checks if a request is allowed using the Fixed Window Counter algorithm."""

    current_time = int(time.time())
    # calculate the start timestamp of the current fixed window
    window_start = math.floor(current_time / WINDOW_SIZE_SECONDS) *
WINDOW_SIZE_SECONDS

    # atomically increment the counter for this window key
    count = STORE.increment(f"{key}:{window_start}", default=0)

    # set expiry when the counter is first created for automatic cleanup
    if count == 1:
       STORE.expire(window_key, WINDOW_SIZE_SECONDS + 5) # Expire 5s after window
ends

    # check if the request is within the rate-limit
    if count > limit:
        return False # Reject
    else:
        return True  # Allow
```

Pros

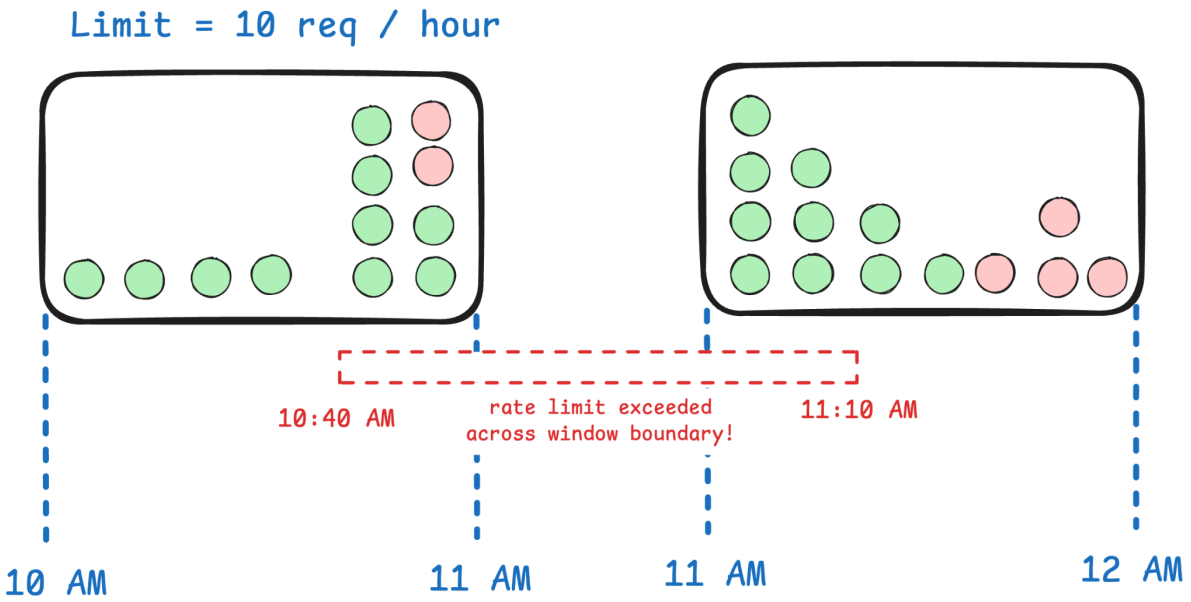
Very simple logic, fast execution, minimal memory (one counter per key for active window).

(user_id, count)
8b 8b = 16 bytes / entry

If you have 2 B users, and you're rate limiting by the user, then you just need 32GB RAM to store all the rate limiting counts

Cons

- Allows bursts at window edges.
- Can result in `2 * limit` requests being processed in a very short time across window boundaries.

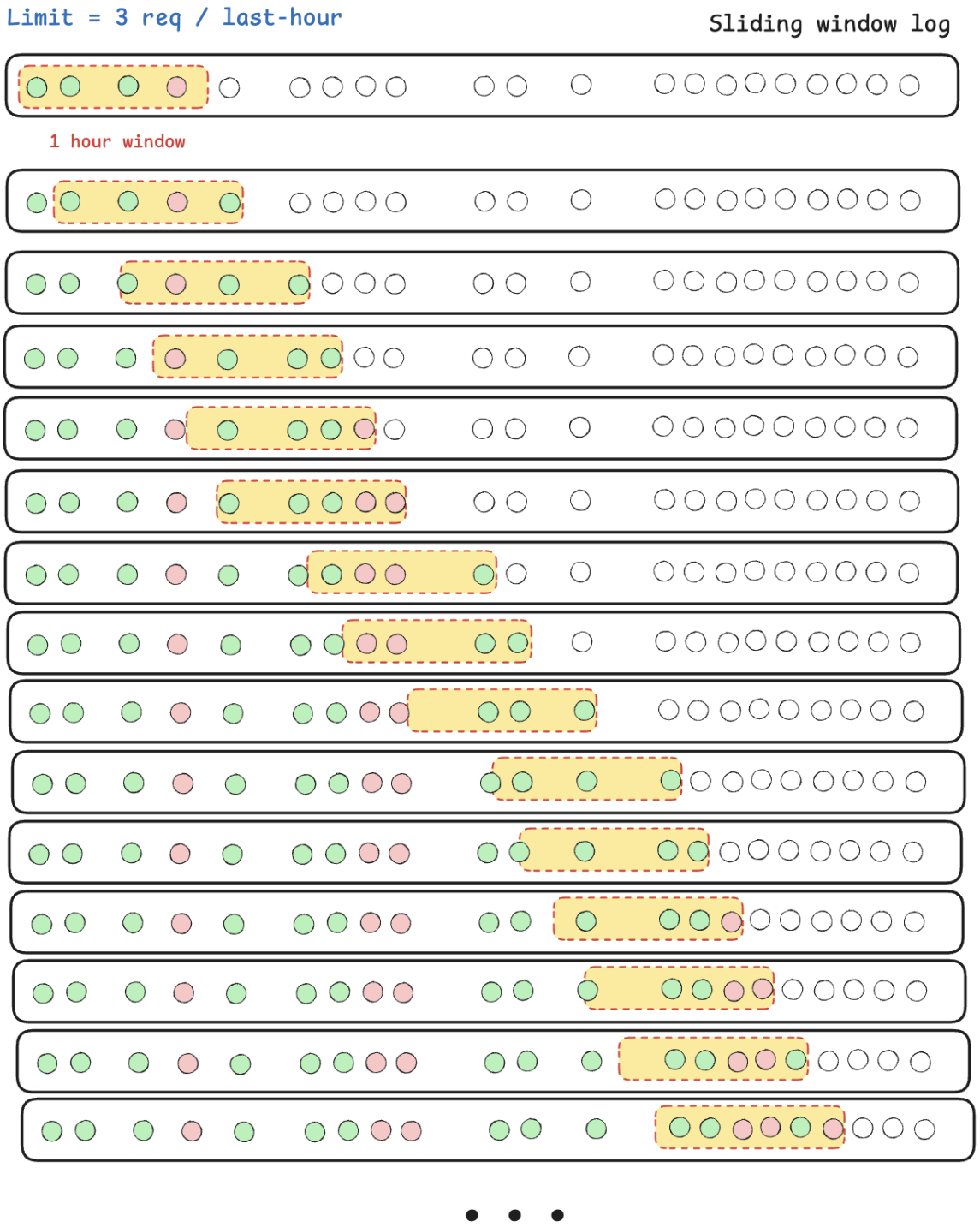


Sliding Window Log

Accurately enforce limits over a true *rolling* time window (eg, `no more than 10 requests in the last 60 seconds`), eliminating the edge burst issue of the Fixed Window method.

Approach

- Maintain a log (queue) of the timestamps for each request received for a given key.
- When a new request arrives
    - remove old timestamps outside the rolling window
    - count the remaining logs
    - allow if count < limit
    - log the new timestamp



Pseudocode

```
import time

# setting rate limit to 3 requests / hour
LIMIT = 3
WINDOW_SIZE_SECONDS = 3600

# Assume STORE provides atomic methods for sorted set manipulation per key
```

```python
# eg, Redis ZSET
def is_allowed_sliding_log(key: str) -> bool:
    """Checks if allowed using Sliding Window Log (conceptual)."""

    current_time = int(time.time() * 1000)
    window_start = current_time - (WINDOW_SIZE_SECONDS * 1000)

    # remove timestamps older than the window start time
    STORE.zremrangebyscore(key, "-inf", window_start)

    # get current count of timestamps within the window
    current_count = STORE.zcard(key)

    # check if the request is within the rate-limit
    if current_count < limit:
        # Add the new timestamp to the log
        STORE.zadd(key, current_time, current_time)
        return True # Allow
    else:
        return False # Reject
```

Pros

Highly accurate for rolling windows, completely smooths edge bursts.

So you can still have bursts within the window, but these bursts will remain within your limit.
For example, if your rate limit is 100 req/hour, then all those 100 requests might arrive in the same second (burst), but no matter what, if you pick any hour, then the number of requests approved within that hour will definitely be <= 100

Cons

- Memory intensive (stores all timestamps in the window per key)
- potentially higher CPU/latency for set/list operations.

Imagine that your rate limits are very large (10,000 requests / hour) then for each user you have to potentially store up to 10,000 timestamps in redis.

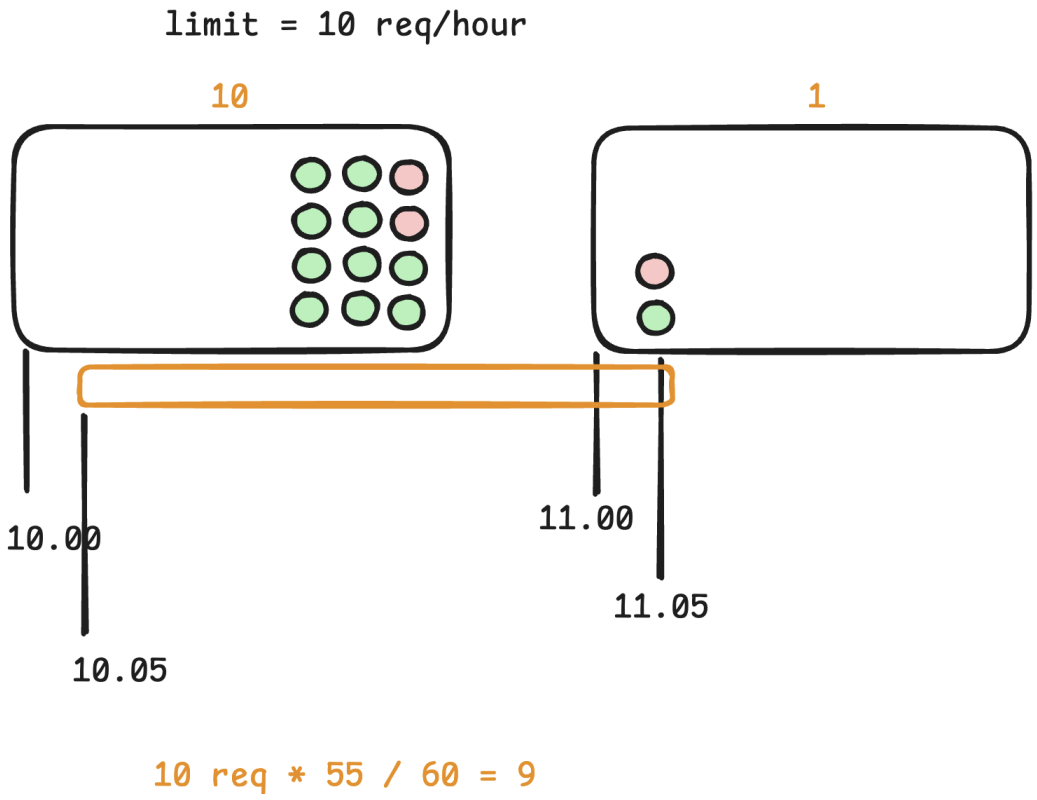
Sliding Window Counter

We want better smoothing than Fixed Window but less memory/CPU overhead than Sliding Window Log.

Approach

- Approximate the rolling window count.
    - Track counts in the *current* and *previous* fixed windows.

- Estimate the contribution from the previous window based on the current time's position within the current window using a weighted calculation.



Pseudocode

```
import time
import math

# setting rate limit to 10 requests / min
LIMIT = 10
WINDOW_SIZE_SECONDS = 60

# Assume STORE is key-value store with atomic operations
# eg, Redis
def is_allowed_sliding_counter(key: str) -> bool:
    """Checks if allowed using Sliding Window Counter approximation."""

    current_time = int(time.time())
    window_start = math.floor(current_time / WINDOW_SIZE_SECONDS) *
WINDOW_SIZE_SECONDS
    previous_window_start = window_start - WINDOW_SIZE_SECONDS

    window_key = f"{key}:{window_start}"
    previous_window_key = f"{key}:{previous_window_start}"
```

```
    # Atomically get both counts
    counts = STORE.mget([previous_window_key, current_window_key])
    previous_count = STORE.get(previous_window_key, 0)
    current_count_before_incr = STORE.get(current_window_key, 0) # Fetch
before potential increment

    # Calculate weight for the previous window's contribution
    time_in_current_window = current_time - window_start
    previous_window_weight = (WINDOW_SIZE_SECONDS - time_in_current_window)
/ WINDOW_SIZE_SECONDS
    previous_window_weight = max(0, previous_window_weight) # Ensure
non-negative

    # Estimate count in the rolling window
    estimated_count = (previous_window_weight * previous_count) +
current_count_before_incr

    if estimated_count >= limit:
        return False # Reject
    else:
        # Allow: Atomically increment current counter & set appropriate
expiry
        new_count = STORE.increment(current_window_key)

        if new_count == 1:
             # Expire after 2 windows to keep previous count available
             STORE.expire(current_window_key, window_size_seconds * 2 + 5)
        return True # Allow
```

Pros

Good balance, smoother than Fixed Window, much lower memory than Sliding Log, good performance.

Cons

It's an approximation, less accurate than Sliding Log under certain burst conditions.

Token Bucket

For each user, maintain the number of tokens they've been awarded.
When a request comes, "consume" 1 token from that's user's bucket.
Periodically (at a fixed rate, say 10 per day) keep "topping-up" the bucket with additional tokens.

Allows for burstiness, but this ensures that the user consumes a fixed “quota” of resources that they’ve paid for.

Eg, for a video processing pipeline, you might purchase “10 credits” which you can use whenever you want. You might add a subscription that gives you 2 additional credits every week.

Leaky Bucket

Used to protect downstream services from burstiness.
(same as the shock absorber idea that I told you about in earlier classes)
As the requests come, they fill up a bucket/buffer.
The downstream service consumes (water leaks from bucket) at a fixed rate.
This ensure that the downstream service always sees a smooth traffic flow.

If the client keeps sending the requests at a high rate for a long period of time, the bucket might “overflow” and those requests will be denied.

Choosing the Right Rate Limiting Algorithm

No single algorithm fits all scenarios. The choice depends on your specific goals

- **Strict Quotas/Period:** Fixed Window (simple) or Sliding Window Log (accurate) or Sliding Window Counter (mix)
- **Average Rate + Bursts:** Token Bucket
    - ChatGPT gives you a fixed number of “messages per day”
    - This message count does NOT reset when the next day behind – it rolls over
- **Fixed Output Rate for downstream protection:** Leaky Bucket

Often, **multiple strategies are layered** across different parts of the infrastructure (eg, Token Bucket/Window Counters @ API Gateway + Leaky Bucket @ Application)

Hardware vs. Software Rate Limiting

Software Rate Limiters

Implemented in application code, middleware, proxies (Nginx, Envoy), gateways. They run on standard servers.

- **Pros:** Highly flexible, lower cost, easy to update.

- **Cons:** Performance limited by server resources, can add latency.

Hardware Rate Limiters

Dedicated network appliances (high-end LBs, Firewalls) using specialized silicon (ASICs/FPGAs).

- **Pros:** Extremely high performance, very low latency.
- **Cons:** Expensive, less flexible configuration. Usually deployed at the network edge.

Rate Limiting at Different OSI Layers

Limiting can happen at various levels

- **Layer 7 (Application):** Most flexible (HTTP details, User ID, API Key). Uses algorithms like Token Bucket, Sliding Window. (Apps, Gateways).
- **Layer 4 (Transport):** IP/Port based (Connection rates, SYN floods). (Firewalls, L4 LBs).
- **Layer 3 (Network):** IP based only (Packet rates). (Routers, Network ACLs, DDoS mitigation infra).

DDoS Attacks

Rate limiting is a **crucial part** but **not a complete solution** for DDoS defense.

- **Helps:** Mitigates application-layer (L7) attacks and some volumetric attacks by capping source rates.
- **Limitations:** Can be overwhelmed by massive network floods; distributed attacks can bypass simple per-IP limits.
- **Requires Layered Defense:** Combine with specialized DDoS mitigation services (Cloudflare, AWS Shield), WAFs, network filtering, IP reputation, CAPTCHAs etc.

Doubts